

2022-2023 学年期末考试真题参考答案

说明：本答案按题面顺序整理，尽量给出可直接复习使用的解释。个别题目若与本学期讲授范围略有出入，可将其看作扩展了解内容。原卷判断题中有一个重复编号的 34，这里仍按原题面顺序作答。

一、判断题

1. 对。图形处理运算精度主要取决于内部数据表示、算法和数值精度，不直接取决于显示器分辨率。分辨率影响的是显示效果，而不是内部运算精度。
2. 对。彩色显示器最常用的是 RGB 颜色模型，因为显示器本身以红、绿、蓝三基色发光。
3. 错。DXF 是 AutoCAD 的图形交换文件格式，不是 ISO 批准的国际图形标准。
4. 对。图形文件系统中，点、线、圆等图元通常用几何参数描述，如端点、圆心、半径等。
5. 对。逻辑输入设备是功能抽象，一个逻辑输入设备可以由多个物理设备实现，例如“定位器”既可由鼠标实现，也可由数位板实现。
6. 错。光栅扫描显示器中的图形通常要经过扫描转换，再在像素阵列上显示，不是按矢量直接描绘。
7. 错。一般应说 Bresenham 算法是对 DDA 一类增量思想的改进，它避免了浮点运算，更适合整数递推。
8. 错。在 Sutherland-Hodgman 逐边裁剪中，若边 $S \rightarrow P$ 的两个端点都在可见侧，通常只输出终点 P ，而不是同时输出 S 和 P 。
9. 对。若共享顶点的两条边落在扫描线同一侧，这个顶点按两个交点处理，局部极值点正属于这种情况。
10. 对。提高分辨率相当于缩小像素尺寸，通常能减轻锯齿，只是代价较高，且并非唯一的反走样方法。
11. 对。把像素看作一个平面区域做面积采样，是反走样的基本思路之一。
12. 错。增强亮度不能消除采样引起的锯齿，只是改变显示强弱。
13. 错。参数表示法的优越性之一恰恰在于不依赖某个坐标轴方向，表达更灵活，并不是“与坐标轴相关使表示更直观”。
14. 错。齐次坐标恰好可以表示无穷远点。
15. 错。平面曲线的判别条件与挠率有关，而不是“曲率为 0”。曲率为 0 反而对应直线情形。
16. 对。若两段曲线在连接点位置相同，切矢方向相同且大小相等，则它们至少达到 C^1 连续，因此也满足 G^1 连续。
17. 对。把 Bezier 曲线的控制顶点顺序完全倒置，曲线形状不变，但参数方向相反。
18. 对。Bezier 曲线具有仿射不变性，因此与坐标系选择无关。
19. 对。Bezier 曲线具有对称性，控制多边形反向时曲线几何形状保持不变。
20. 错。题目描述的是**变差缩减性**，不是凸包性。凸包性指曲线位于控制多边形的凸包内。
21. 错。当结点向量退化为两端重复结点时，对应情形更接近 Bezier 形式，并不会简单退化成直线。
22. 错。B 样条节点插入后，结点矢量和控制点会改变，但曲线形状保持不变。

- 23. 错。NURBS 的重要优点之一，就是能统一表示标准解析曲线与自由曲线、曲面。
- 24. 对。NURBS 可以通过调节控制顶点和权因子来改变形状。
- 25. 错。NURBS 不仅具有仿射不变性，还具有透视不变性。
- 26. 对。NURBS 比普通均匀 B 样条具有更强的表达能力，能表示更多标准解析形状。
- 27. 错。对不等距数据点，NURBS 往往更灵活，但不能简单断言“一定更光滑”。
- 28. 对。权因子控制提供了除移动控制点之外的另一种有效形状调节手段。
- 29. 错。NURBS 主要提升的是表达统一性和造型能力，不是直接提高曲面显示效率。
- 30. 对。权因子选择不合理时，NURBS 曲线确实可能出现畸变。
- 31. 对。NURBS 可以表示圆锥曲线等标准解析形状，也可以表示自由曲线曲面。
- 32. 对。双三次参数曲面片几何系数矩阵中的相关子阵可用于表示角点处的扭矢 (twist vectors)。
- 33. 对。平移变换不改变图形的大小和形状，只改变位置。
- 34. 错。错切会改变角度关系，通常会引入形状畸变。
- 35. 对。旋转变换保持长度、角度和线性关系不变，属于刚体变换。
- 36. 对。在三维透视投影中，主灭点个数最多为 3 个。
- 37. 对。画家算法的基本思想就是按深度次序从远到近绘制，各面逐次覆盖前面的背景或较远物体。
- 38. 对。Z 缓冲算法同时需要颜色缓冲和深度缓冲。
- 39. 错。若投影包围盒不相交，只能说明屏幕上没有重叠区域，因此不存在遮挡关系，不能据此推出线段“更靠近视点”。
- 40. 对。区域采样消隐算法正是利用区域连贯性，在区域层面确定可见面及其颜色亮度。

二、简述你对 DirectX 的理解

DirectX 是微软提出的一组多媒体与图形编程接口集合，主要服务于 Windows 平台下的图形、声音和输入等应用开发。它并不只是一个“画图函数库”，而是一整套面向实时图形和交互式程序的底层接口体系。

在图形学语境下，最常被提到的是 Direct3D，它负责三维图形绘制，包括顶点处理、光栅化、纹理映射、着色器等现代图形流水线功能。除此之外，DirectX 体系还包含与二维绘制、音频、输入设备等有关的组件，因此它常用于游戏开发、实时可视化和交互式多媒体程序。

从作用上看，DirectX 的意义在于为应用程序提供统一而高效的硬件访问方式，使软件可以较充分地利用 GPU 和其他多媒体设备能力。与 OpenGL 相比，它更深地绑定微软平台生态；与更高层的引擎相比，它又更接近底层图形接口。

三、画图详细描述上半部分椭圆弧的绘制原理、算法步骤及判别式递推

设椭圆标准方程为

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1$$

对应隐函数可写为

$$F(x, y) = b^2x^2 + a^2y^2 - a^2b^2$$

若要求上半部分椭圆弧，可以从第一象限起点 $(0, b)$ 出发，逐点生成第一象限的椭圆弧，再利用关于 y 轴的对称性补出第二象限，从而得到完整上半椭圆。

中点椭圆算法的核心思想是：每一步只在两个候选像素之间作选择，利用它们中点代入隐函数的符号来判断下一步取哪一个像素。由于椭圆斜率在不同区域变化不同，通常要分成两个区域处理。

当椭圆上点满足 $|dy/dx| < 1$ 时，说明曲线较平缓，优先沿 x 方向推进；当 $|dy/dx| > 1$ 时，说明曲线较陡，优先沿 y 方向推进。由

$$\frac{dy}{dx} = -\frac{b^2x}{a^2y}$$

可知分界条件为

$$b^2x = a^2y$$

区域 1 的算法

区域 1 从 $(0, b)$ 开始，满足 $b^2x < a^2y$ 。候选点为东点 $E(x+1, y)$ 和东南点 $SE(x+1, y-1)$ ，取它们中点

$$M\left(x+1, y-\frac{1}{2}\right)$$

定义判别式

$$p_1 = F\left(x+1, y-\frac{1}{2}\right)$$

初值在 $(0, b)$ 处为

$$p_1^{(0)} = b^2 - a^2b + \frac{a^2}{4}$$

若 $p_1 < 0$ ，说明中点在椭圆内，真实曲线更靠近 E ，下一点取 E ，即

$$x_{k+1} = x_k + 1, \quad y_{k+1} = y_k$$

若 $p_1 \geq 0$ ，说明中点在椭圆外或边界上，真实曲线更靠近 SE ，下一点取 SE ，即

$$x_{k+1} = x_k + 1, \quad y_{k+1} = y_k - 1$$

对应的判别式递推不难求得。

区域 2 的算法

当进入 $b^2x \geq a^2y$ 后，转入区域 2。此时曲线较陡，候选点改为南点 $S(x, y - 1)$ 和东南点 $SE(x + 1, y - 1)$ ，取中点

$$M\left(x + \frac{1}{2}, y - 1\right)$$

定义判别式

$$p_2 = F\left(x + \frac{1}{2}, y - 1\right)$$

区域 2 的初值由区域 1 最后一点代入上式计算得到。

若 $p_2 > 0$ ，说明中点在椭圆外，取南点 S ，即

$$x_{k+1} = x_k, \quad y_{k+1} = y_k - 1$$

若 $p_2 \leq 0$ ，说明中点在椭圆内，取东南点 SE ，即

$$x_{k+1} = x_k + 1, \quad y_{k+1} = y_k - 1$$

对应的判别式递推也不难求得。

算法步骤概括

1. 设初始点为 $(0, b)$ 。
2. 在区域 1 内，依据 p_1 的符号在 E 与 SE 之间选择。
3. 当满足 $b^2x \geq a^2y$ 时转入区域 2。
4. 在区域 2 内，依据 p_2 的符号在 S 与 SE 之间选择。
5. 生成第一象限后，利用关于 y 轴对称补出第二象限，即得上半部分椭圆弧。

四、回答下列问题

(1) 什么叫 Bezier 曲线的对称性与凸包性

对称性: 若一条 Bezier 曲线的控制顶点顺序为 $P_0, P_1, \dots, P_n$, 将控制顶点顺序完全颠倒为 $P_n, P_{n-1}, \dots, P_0$ 后, 得到的新曲线与原曲线几何形状完全相同, 只是参数方向相反。这就是 Bezier 曲线的对称性。

更形式化地说, 若原曲线为

$$P(t) = \sum_{i=0}^n P_i B_{i,n}(t)$$

则将控制点反序后得到的曲线与原曲线满足

$$\tilde{P}(t) = P(1-t)$$

所以几何轨迹不变。

凸包性: Bezier 曲线始终位于其控制顶点构成的凸包内部, 不会跑到这个凸包外面去。这一性质在作图上意味着: 只要画出控制多边形, 就能较直观地估计曲线大致处于什么范围。

(2) B 样条基函数的局部性

这一问的关键是“局部性”。设曲线为 p 次 B 样条曲线, 则在参数区间 $[t_i, t_{i+1}]$ 上, 只有在该区间内基函数不为零的那 $p+1$ 个控制顶点会影响曲线。

也就是说, 在区间 $[t_i, t_{i+1}]$ 上, 曲线只与

$$P_{i-p}, P_{i-p+1}, \dots, P_i$$

这 $p+1$ 个控制顶点有关。

因此, 这一小段曲线也必定落在上述这些顶点构成的凸包内部。

进一步地, 某个控制顶点 P_i 并不会影响整条曲线, 而只影响一段有限参数区间。对 p 次 B 样条而言, P_i 影响的参数范围为

$$[t_i, t_{i+p+1}]$$

这就是 B 样条局部可控性的来源: 只动一个控制点, 只会改动曲线的局部, 而不会像高次 Bezier 那样牵动整条曲线。

五、回答下列问题

(1) 证明两条参数曲线达到 C^1 和 G^1 连续, 并求连接点

两条曲线分别为

$$r(t) = (-t^2 + 2t, t)$$

和

$$\eta(t) = (t^2 + 1, t + 1)$$

设它们在参数 $t = t_1$ 和 $t = t_2$ 处连接, 则先要求位置相同。

由第二个分量得

$$t_1 = t_2 + 1$$

代入第一个分量,

$$-t_1^2 + 2t_1 = t_2^2 + 1 = (t_1 - 1)^2 + 1$$

化简得

$$-t_1^2 + 2t_1 = t_1^2 - 2t_1 + 2$$

即

$$2t_1^2 - 4t_1 + 2 = 0$$

所以

$$(t_1 - 1)^2 = 0$$

从而

$$t_1 = 1, \quad t_2 = 0$$

连接点为

$$r(1) = \eta(0) = (1, 1)$$

再求导数:

$$r'(t) = (-2t + 2, 1)$$

$$\eta'(t) = (2t, 1)$$

于是

$$r'(1) = (0, 1), \quad \eta'(0) = (0, 1)$$

两条曲线在连接点处一阶导向量完全相同，因此达到 C^1 连续。

由于 C^1 连续必然推出 G^1 连续，所以它们也达到 G^1 连续。

结论：两条曲线在点 $(1, 1)$ 处达到 C^1 连续，也达到 G^1 连续。

(2) 两段五次参数曲线达到 C^3 连续的条件

设两段五次参数曲线分别为 $P_1(u)$ 和 $P_2(v)$ ，若它们在 $u = 1$ 和 $v = 0$ 处拼接，则达到 C^3 连续的条件是：

$$P_1(1) = P_2(0)$$

$$P_1'(1) = P_2'(0)$$

$$P_1''(1) = P_2''(0)$$

$$P_1'''(1) = P_2'''(0)$$

用文字说，就是在连接点处，两段曲线的位置、一阶导矢、二阶导矢和三阶导矢都分别相等。因为五次曲线自由度较高，所以常用来满足较高阶连续条件。

六、简答题

1. 简述 MIT 在计算机图形学早期发展时做出的重要贡献

MIT 在计算机图形学早期发展中具有奠基性作用。重要贡献主要包括：

1. 较早开展交互式图形学研究，把计算机从单纯数值计算工具推进为可交互的图形处理平台。
2. 在 Whirlwind、SAGE 等系统背景下推动了实时显示与交互设备的发展。
3. Ivan Sutherland 在 MIT 开发的 Sketchpad 被普遍视为现代交互式图形系统和 CAD 的重要起点之一，它引入了图形交互、约束、层次结构等关键思想。
4. MIT 的相关工作带动了后续 CAD、人机交互和图形软件系统的发展，对整个学科方向的形成影响很大。

2. CGM 是什么？与 GKSM 的区别是什么？

CGM 是 **Computer Graphics Metafile**，即计算机图形元文件标准。它的目标是用与设备无关的方式记录图形信息，便于不同系统之间交换与存储图形。

GKSM 一般指 **GKS Metafile**，即图形核心系统 GKS 的元文件。它更偏向 GKS 系统内部或与 GKS 配套的图形记录方式。

二者区别可以概括为：

1. CGM 是更通用、更标准化的图形交换格式。
2. GKSM 更贴近 GKS 图形标准体系。
3. CGM 面向更广泛的图形交换与存档，而 GKSM 更强调 GKS 环境中的图形记录。

3. 画图并文字描述事件方式的交互输入过程

事件方式交互的核心思想是：系统平时处于等待状态，只有当用户触发某种事件时，才调用相应处理程序。

其基本流程可描述为：

1. 用户通过鼠标、键盘等设备产生输入。
2. 设备驱动把输入转换为事件，如鼠标单击、按键按下、窗口重绘等。
3. 事件进入事件队列。
4. 应用程序主循环不断从事件队列中取出事件。
5. 根据事件类型分发给相应的事件处理函数。
6. 处理函数修改程序状态或图形对象。
7. 系统刷新显示，用户看到新的结果。

若画图，可画成：

用户操作 -> 输入设备 -> 事件队列 -> 事件分发 -> 处理程序 -> 图形更新 -> 用户感知反馈

4. 简述轮廓字型法的基本原理

轮廓字型法不是把字符表示成固定点阵，而是用字符外轮廓的几何边界来描述字形，这些边界通常由直线段、Bezier 曲线或样条曲线组成。

其基本原理是：

1. 先用曲线和线段描述字符轮廓。

2. 显示或打印时，再对轮廓内部进行填充。

3. 由于本质上保存的是几何轮廓，因此字符可以方便地缩放、旋转、变换，而不易失真。

相比点阵字，轮廓字的优点是分辨率独立、缩放后仍清晰、适合高质量排版和打印。因此它在中英文排版、出版印刷、PostScript、TrueType 等字型系统中都非常常见。

5. B 样条的节点插入算法技术目的和特点是什么？

节点插入的技术目的，主要是为了在**不改变曲线形状**的前提下，增加控制能力和局部调整能力。

它的主要特点包括：

1. 插入节点后，曲线几何形状保持不变。
2. 结点矢量和控制顶点会发生变化。
3. 曲线的局部表示会更细，有利于局部修改、细分和后续计算。
4. 节点重复次数增加时，还会影响连接处连续性。

因此，节点插入常被用于曲线细化、局部编辑和曲线拼接处理。

6. 写出关于 (x_f, y_f) 点的缩放变换矩阵

若相对于固定点 (x_f, y_f) 做缩放，缩放系数分别为 s_x 和 s_y ，则齐次坐标下的二维变换矩阵为

$$\begin{bmatrix} s_x & 0 & x_f(1 - s_x) \\ 0 & s_y & y_f(1 - s_y) \\ 0 & 0 & 1 \end{bmatrix}$$

它的含义是：先把固定点平移到原点，做缩放，再平移回去。

7. 四叉树消隐算法中，窗口处于何种情况下称为足够简单可以直接显示？

所谓“足够简单”，本质上是指当前区域内的可见关系已经明确，不必再继续细分。典型情形包括：

1. 区域内没有任何图形。
2. 区域完全被某一个可见面覆盖。
3. 区域内虽然有多个面，但已经可以确定最终可见面唯一，颜色和亮度也可直接确定。
4. 区域中不再存在需要继续判断的边界交叉、遮挡歧义等复杂情况。

这时就可以直接对该区域赋值显示，而不必继续递归分割。

8. VR 系统中对语音与声音有什么要求?

VR 中的声音不只是“能播放出来”即可，而是要满足沉浸式环境下的感知要求，主要包括：

1. **空间性**：声音方向和位置要与虚拟场景一致，用户转头后，声源方向应相应变化。
2. **同步性**：声音与画面、动作、交互反馈要尽量同步，不能明显错位。
3. **低延迟**：声音更新不能太慢，否则会破坏沉浸感。
4. **真实性与环境一致性**：应考虑距离衰减、回声、遮挡等环境因素，使听感更接近真实空间。

9. 画图说明函数 void glLineStipple(2, 0x3f07) 及其参数的意义

glLineStipple 用于设置线型模式，它的两个参数分别是：

1. 第一个参数 2 是重复因子 (factor)，表示模式中的每一位都重复 2 次。
2. 第二个参数 0x3f07 是 16 位位模式 (pattern)，其中比特位为 1 表示画线，为 0 表示空白。

0x3f07 写成二进制是

0011111100000111

OpenGL 按低位到高位使用该模式，因此实际输出顺序可理解为

1110000011111100

再乘以重复因子 2，相当于每一位延长为两个像素单位，于是形成一条按固定模式重复出现的虚线。概念上可描述为：先画一段短实线，再留一段空白，再画一段较长实线，再留一段较短空白，然后循环。

10. OpenGL 中设置光源的函数是什么？挑选三个光源特性参数说明其含义

OpenGL 中设置光源常用的函数是 glLightf、glLightfv、glLighti、glLightiv 这一组，其中最常见的是 glLightfv。

例如：

```
glLightfv(GL_LIGHT0, GL_POSITION, pos);
```

常见光源特性参数可以举以下三个：

1. GL_POSITION：设置光源位置。若第四个分量为 1，表示点光源；若为 0，表示方向光。
2. GL_DIFFUSE：设置光源的漫反射颜色强度，决定物体在受光面上的主要亮度贡献。
3. GL_SPECULAR：设置镜面反射颜色强度，决定高光效果的强弱和颜色。

此外, `GL_AMBIENT` 用于设置环境光分量, `GL_SPOT_DIRECTION` 用于设置聚光灯方向, 也都很常见。